# Scalable web application assessment

## Architecture, implementation guide and customer review

Prepared for Krishna Aenikapati | 23 October 2026

A complete implementation package for a small stateless application on DigitalOcean Kubernetes. Includes container packaging, autoscaling configuration, deployment scripts, a beginner setup guide and a two-page QBR draft.

| Completion status | Evidence |
| --- | --- |
| Prepared and locally checked | Application, configurations, scripts and documentation |
| Local tests executed | 6 automated tests passed; short HTTP smoke test passed |
| Cloud execution pending | Image build, DOKS deployment, scaling and recovery evidence |
| Publication pending | A dedicated GitHub repository is required |

This document labels pending work explicitly. The local benchmark is not a DigitalOcean performance result.

# Customer objectives and design

| Customer goal | Design response | How to verify |
| --- | --- | --- |
| Scalability | CPU-based HPA, 2-6 application pods | Record replica changes under bounded load |
| Performance | Resource requests/limits and workload measurements | Record successful throughput, p95 and failures |
| Reliability | Multiple replicas, health checks and rolling updates | Observe pod replacement and request continuity |
| Cost control | Small baseline and optional bounded node scaling | Compare verified account rates and actual capacity |

## Scope

The application is a stateless Node.js service with no third-party application packages. Its homepage displays the responding instance. A token-protected CPU endpoint enables repeatable scaling tests. A generated Kubernetes Secret provides the test token.

## Design boundaries

This lab uses one region and HTTP. Production recommendations include HTTPS, operational monitoring, access controls, image scanning and business-specific recovery planning. Multiple pods improve resilience to selected failures but do not establish an availability SLA.

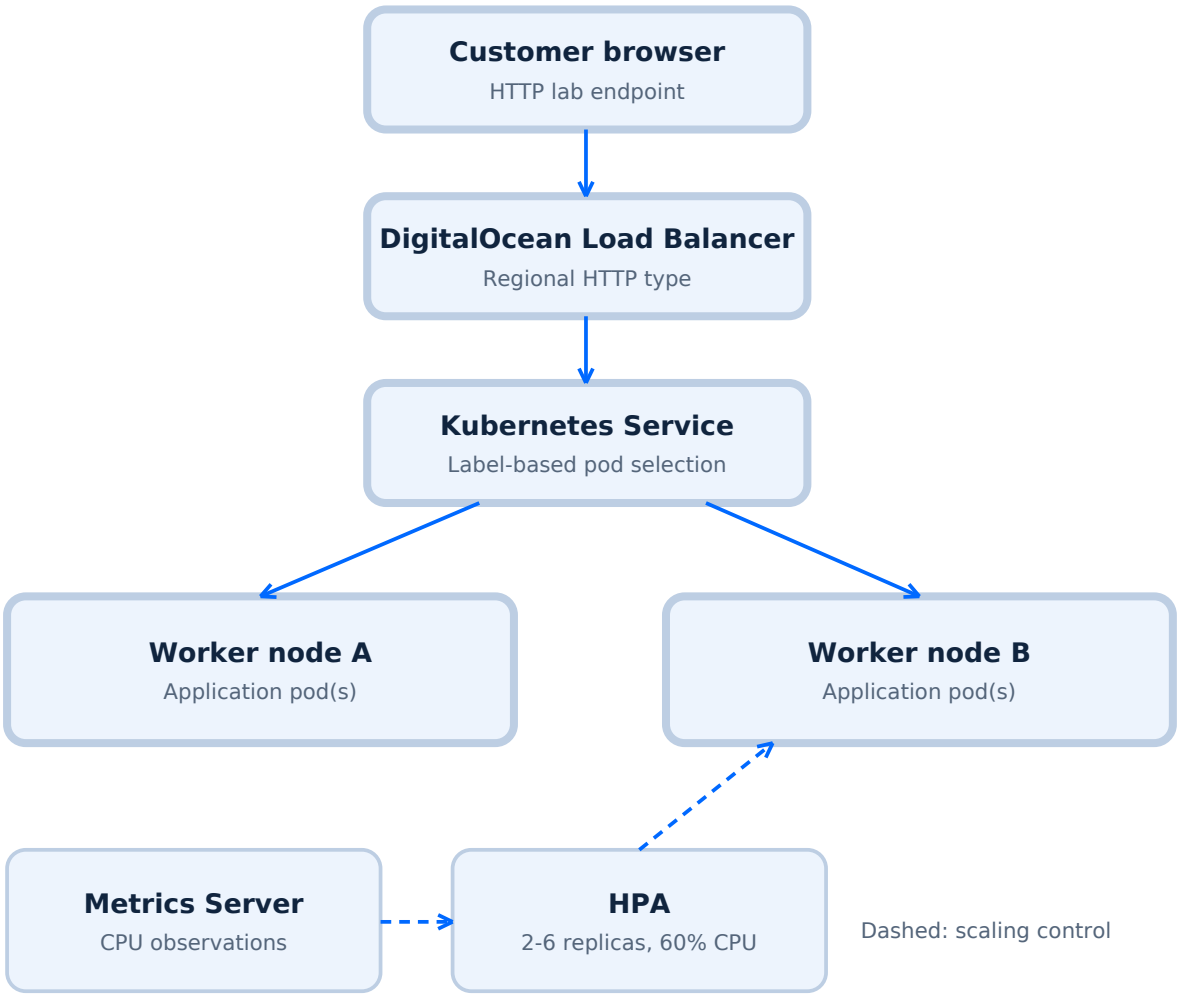
## Local verification

The local loopback smoke test completed 387 requests in 3.01 seconds at concurrency 2, with 0 failures. Its p95 was 18.64 ms. These measurements verify the test path in the authoring runtime only. They do not predict cloud capacity.

## Submission checklist

Publish the dedicated repository, deploy the container, verify the public endpoint, collect HPA and recovery evidence, replace pending results and example prices, then submit the repository link and updated PDF.

# Architecture




The registry stores the Docker image. The Deployment pulls it and maintains application pods. The diagram shows request routing and HPA control; it does not imply that the load balancer contacts pods without Kubernetes networking.

## Pod scaling and worker scaling

HPA changes pod count based on CPU utilization. Optional cluster autoscaling changes worker count when pods cannot fit. The default six small replicas may fit on two workers. Observe a node change before claiming server scaling.

## Placement and disruption

Soft topology spreading favors separate workers and permits fallback during capacity loss. Verify actual placement. The disruption budget limits supported voluntary evictions and does not prevent abrupt node failure.

# Beginner setup guide

## 1. What you will build

A web application runs in a Docker container. Kubernetes starts multiple copies called pods. Worker nodes are the servers hosting the pods. A public load balancer sends requests into the cluster. HPA watches CPU usage and changes pod count. Metrics Server supplies CPU observations. A registry stores your application image. kubectl is the command you use to manage Kubernetes, while doctl authenticates to DigitalOcean.

Use a laptop or desktop. The commands assume Bash on macOS, Linux, or Ubuntu in Windows WSL2. On Windows, install Docker Desktop with WSL integration and run all commands from the same Ubuntu terminal. You do not need to edit the application code to complete this walkthrough.

## 2. Install the tools

1. Install Docker Desktop from https://docs.docker.com/desktop/ and open it. Wait for the engine to start. On Linux, an official Docker Engine installation is also suitable.

2. Install Node.js 24 LTS from https://nodejs.org/. This runs the local app and tests.

3. Install kubectl from https://kubernetes.io/docs/tasks/tools/. Select the instructions for your operating system. Use a version compatible with the selected cluster.

4. Install doctl from https://docs.digitalocean.com/reference/doctl/how-to/install/.

5. Install Python 3 from https://www.python.org/downloads/. No Python packages are needed for the provided deployment/test scripts.

6. Install Git from https://git-scm.com/downloads and an editor such as VS Code from https://code.visualstudio.com/.

7. Open a new terminal after installation.

8. Run each command below. A version should appear. `docker info` should connect to the engine.

```
node --version
python3 --version
docker --version
docker info
kubectl version --client
doctl version
git --version
```

If a command is missing, finish that installation before proceeding. These tools run on your computer. The Kubernetes servers will run in DigitalOcean.

## 3. Unzip and open the project

Download TAM_Assessment_Package.zip. Extract it. Open the tam-doks-demo folder in your editor. Open a terminal inside that folder. Run `pwd` and `ls` to verify you see Dockerfile, package.json, app, k8s and scripts. All later commands assume this directory.

```
node --test tests/app.test.mjs
node app/server.mjs
```

Open http://localhost:8080 in your browser. Click Refresh response. You should see an instance name and response time. Local response time is not cloud performance. Return to the terminal and press Ctrl+C to stop the server.

## 4. Check Docker locally

```
docker build -t tam-demo:local .
docker run --rm -p 8080:8080 --name tam-demo tam-demo:local
```

Open http://localhost:8080 again. Open http://localhost:8080/healthz and expect status ok. Press Ctrl+C to stop the container. If port 8080 is occupied, stop the earlier server before retrying. The app intentionally has no database and no login screen.

## 5. Prepare DigitalOcean and Docker Hub

1. Sign into DigitalOcean and confirm the assessment credits with the recruiter or account billing page.

2. Create a project for this assessment, keeping it separate from customer systems.

3. In Docker Hub, create a public repository named tam-demo. The project contains demonstration code only.

4. Run `docker login` and complete its secure authentication process.

5. In DigitalOcean, open API and create/use a token with the permissions needed for the intended cluster operations. Keep it private and follow the account's access policy.

6. Run `doctl auth init` and supply the token privately at the prompt. Do not put it in code or screenshots.

7. Run `doctl account get` to confirm which account you are using. Do not publish account output in your evidence.

## 6. Create the cluster in the console

1. Open Kubernetes and select Create Kubernetes Cluster.

2. Choose one region near the expected customer population. For this demo, NYC1 is a reasonable example if available.

3. Choose a currently supported Kubernetes version.

4. Choose an AMD64 basic worker node type around 2 vCPU and 4 GiB RAM. Note the exact hourly/monthly price shown.

5. Set the worker count to two.

6. Name the cluster tam-demo-cluster.

7. Check the estimated recurring cost and available credits, then create the cluster.

8. Wait until both workers are Ready. Optional HA control-plane charges must be included if you select that option.

9. Keep the lab in one region and describe regional recovery as future work.

Alternative CLI creation, only after verifying size and region availability:

```
doctl kubernetes options regions
doctl kubernetes options sizes
```

The GUI route above is sufficient. Do not create a second cluster accidentally by repeating a creation command.

## 7. Deploy using the supplied script

Replace the Docker Hub username below with your own. Keep the cluster name aligned with the cluster you just created.

```
export DOCKERHUB_USER=YOUR_DOCKERHUB_USERNAME
export TAM_CLUSTER_NAME=tam-demo-cluster
bash scripts/deploy.sh
```

The script connects kubectl to the named cluster, builds and pushes an AMD64 image, creates the namespace and private load-test token, applies the Deployment, HPA, disruption budget and Service, and waits for rollout. It prints the target context before changes. Read that line to ensure you are targeting the assignment cluster.

The deployment script does not create a cluster. If the name is wrong, it stops when doctl fails. If Docker push fails, verify the repository and login. The Docker Hub repository must be public for this version of the manifests. Private repositories require an image pull secret.

```
kubectl get pods -n tam-demo -o wide
kubectl get service tam-demo -n tam-demo -w
```

Wait until there is an EXTERNAL-IP, then press Ctrl+C. Open http://EXTERNAL-IP. The Service creates the DigitalOcean load balancer. Do not create another load balancer manually. The explicit REGIONAL annotation selects the HTTP load balancer type.

## 8. Enable CPU metrics

Run `kubectl top nodes`. If CPU and memory values appear, metrics already work. Otherwise, open the cluster's Marketplace/1-Click Apps page, select Kubernetes Metrics Server, and install it. Wait a few minutes and retry.

```
kubectl top nodes
kubectl top pods -n tam-demo
kubectl get hpa -n tam-demo
```

A CPU target such as 5%/60% indicates current average usage and target. A persistent unknown value requires investigation. Check `kubectl describe hpa tam-demo -n tam-demo`. HPA uses CPU requests, so 60% of 100m means approximately 60m per pod. Wait for at least two ready pods before testing.

## 9. Capture live results

Get the external IP from the Service and set the URL. Confirm the current context is your assignment cluster before saving it as the expected context.

```
kubectl config current-context
export APP_URL=http://YOUR_EXTERNAL_IP
export TAM_EXPECTED_CONTEXT="$(kubectl config current-context)"
bash scripts/collect-evidence.sh
```

In a second terminal, watch:

```
kubectl get hpa,pods -n tam-demo -w
```

The script measures the home endpoint, then the CPU workload at low concurrency, then applies 20 concurrent requests for 180 seconds while capturing HPA, pod count and CPU snapshots. Outputs go to a timestamped evidence/cloud-* directory. The script never prints the token. Stop a test with Ctrl+C if it causes unexpected effects.

Record pod count before, peak pod count, time to first new ready replica, throughput, p95 latency, error counts and load-generator location. After load stops, watch for 5-10 minutes and record the

time to return to two pods. Scale-down is not instantaneous. Do not claim node autoscaling if node count never changed.

## 10. Test recovery and updates

Run continuous requests from one terminal while replacing a single pod. Use the actual pod name from `kubectl get pods`.

```
for i in {1..60}; do
  curl --max-time 5 -sS -o /dev/null -w '%{http_code}\n' "$APP_URL/"
  sleep 1
done
```

From another terminal:

```
kubectl get pods -n tam-demo
kubectl delete pod ACTUAL_POD_NAME -n tam-demo
kubectl get pods -n tam-demo -w
```

Record errors and recovery time. This tests pod replacement, not a full node outage. A disruption budget does not protect against direct pod deletion. For an update demonstration, make a visible text change, rerun the deployment script, and observe rollout. If a new version fails, run `kubectl rollout undo deployment/tam-demo -n tam-demo`, then verify `kubectl rollout status`. Preserve evidence from both versions.

## 11. Optional node autoscaling

Open Kubernetes, select the cluster, then Resources. On the node pool menu choose Resize or Autoscale. Set Autoscale, minimum two nodes and maximum three. It adds capacity when pods cannot be scheduled and may remove unnecessary nodes when safe. Six small application pods can fit on the existing workers, so this test may not trigger a new server. Verify actual scheduling and node events before making any claim.

## 12. Cost analysis

Record the exact node plan and load balancer rates from the account. Use the cost script with those values. Example assumptions only:

```
python3 scripts/cost.py --node-monthly 24 --lb-monthly 12 --extra-node-hours 40
```

The example produces a two-worker baseline of $60/month and a three-worker fixed scenario of $84/month. Keeping the third worker for only 40 of an assumed 730 hours gives about $61.32/month, a modeled difference of $22.68. This is a scenario, not measured savings. Include all other charges. A fixed two-node cluster costs the same at two and six pods.

## 13. Publish and submit

Create a dedicated GitHub repository named tam-doks-demo. Use public visibility only if the assessment permits it. Initialize an empty repository, then run the following from the project directory after reviewing the files:

```
git init
git add .
git commit -m "Add DigitalOcean TAM assessment"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/tam-doks-demo.git
git push -u origin main
```

Use secure GitHub authentication. Do not commit tokens, kubeconfig or unreviewed logs. Add reviewed cloud results, update the QBR and cost figures, and regenerate the PDF if the results

change. Submit the actual repository URL and final PDF to the assessor. Do not submit the initial package as if deployment had already occurred.

## 14. Troubleshooting

| Symptom | Action |
|---|---|
| Docker daemon unavailable | Open Docker Desktop, wait for engine readiness, rerun docker info |
| ImagePullBackOff | Verify public repository, correct tag and pull events in kubectl describe pod |
| Exec format error | Build the image for linux/amd64 to match the chosen workers |
| HPA target unknown | Install Metrics Server, wait, confirm CPU requests and inspect HPA conditions |
| Pods Pending | Describe the pod, inspect resources, quota and node capacity |
| EXTERNAL-IP pending | Inspect Service events and DigitalOcean quota/provisioning status |
| HTTP 403 on /work | Use collect-evidence.sh so it retrieves the authorized test token |
| HTTP 429 during testing | Per-pod work cap reached; inspect errors and reduce concurrency |
| No scale-up | Verify /work requests succeed, CPU rises and the test runs long enough |
| No scale-down yet | Allow the five-minute stabilization window and inspect HPA events |
| Missing second worker placement | Check pods -o wide and topology constraints; soft spreading allows fallback |

## 15. Cleanup

Save and review evidence, then confirm that the assessor no longer needs a live URL. Verify kubectl's context. Delete the Service, wait for the managed load balancer to disappear, then delete the namespace. Finally delete only the assignment cluster in the DigitalOcean console and inspect for leftover volumes or load balancers. Stop paying for the infrastructure, not just the pods.

```
kubectl delete service tam-demo -n tam-demo
kubectl delete namespace tam-demo
```

# QBR summary / 1 of 2

Prepared for the assessment scenario by Krishna Aenikapati. Review date: 23 October 2026. Status: implementation package prepared; cloud validation pending. This is a pre-deployment review draft, not a claim of completed customer production work.

## Customer objectives and current position

The customer needs to onboard more users while maintaining responsive service, minimizing disruption and avoiding unused infrastructure. The proposed foundation is a stateless Node.js application on DigitalOcean Kubernetes with a public regional HTTP load balancer, two baseline application replicas and CPU-based horizontal scaling up to six pods.

Two worker nodes provide separate placement opportunities. Soft topology spreading favors distinct hosts without blocking emergency scheduling. Health probes remove unready pods from service and restart unhealthy containers. Rolling updates retain available replicas when sufficient capacity exists. A disruption budget preserves one available replica during supported voluntary evictions. These controls reduce specific failure risks but do not establish a contractual availability level or regional disaster recovery.

## Evidence available

Six automated local tests passed: homepage, health/readiness, instance identity, unauthorized load rejection, authorized CPU work, and invalid request handling. A short local HTTP smoke test is included in the evidence folder. Container build, live DOKS deployment, external load balancing, HPA scale-up/down and recovery measurements remain pending.

## Customer acceptance measures

Record the deployed image, cluster version and node plan. Capture the public endpoint, two ready replicas and placement on separate workers. Run the fixed-duration load test and record p95 response time, successful requests per second, failures and replica counts. Observe scale-down after load ends. Delete one pod while issuing requests and record replacement time and any interruption. Agree workload-specific latency and availability targets with the customer before assigning a pass/fail threshold.

## Cost position

Use the actual account quote for final analysis. Under example assumptions of $24 per worker per month and $12 for the load balancer, two workers cost $60/month before optional services, usage overages and taxes. Three fixed workers cost $84/month. Keeping a third worker for 40 of 730 modeled hours gives approximately $61.32/month. The $22.68 difference is modeled capacity savings relative to three fixed workers, not observed savings. Pod scaling alone does not reduce billed worker count.

# QBR summary / 2 of 2

## Recommendations

First, complete the cloud validation and review evidence with the customer. Choose resource requests and maximum replicas from measured throughput and latency. If scheduling pressure appears, enable bounded node autoscaling with a minimum of two and maximum of three workers. After the lab, remove the CPU test endpoint or keep it behind an authenticated testing boundary.

Before production, introduce HTTPS and a domain, monitoring and actionable alerts, image scanning and digest pinning, least-privilege access, and a documented rollout and rollback procedure. Assess database backup and resilience needs if a stateful service is added. Decide whether regional recovery is justified by the customer's recovery objectives and budget.

## Risks and mitigations

| Risk | Customer impact | Mitigation |
|---|---|---|
| Burst arrives before scaling completes | Temporary latency or errors | Maintain baseline capacity and test expected bursts |
| Metrics unavailable | HPA cannot make normal CPU scaling decisions | Alert on missing metrics and monitor HPA conditions |
| Resource limit too low | Throttling or memory restarts | Right-size against measurements and review trends |
| Region outage | Application unavailable | Agree recovery objectives and evaluate a second region |
| Unused resources remain | Continued charges | Assign cleanup ownership and review resource inventory |
| HTTP demonstration | Unencrypted requests and test token | Use synthetic data, short lab duration and HTTPS before production |

## Action plan

The assessment owner completes account access and deployment first. The engineering owner collects load, recovery and cost evidence immediately after deployment. The customer service owner reviews the results against agreed business targets. Before production approval, the security and operations owners complete HTTPS, monitoring and access controls. Each action needs a named owner and calendar date in the final customer version.

## Decision requested

Confirm the expected peak workload, target response time, acceptable recovery time, monthly cost envelope and whether the application must survive a region outage. These inputs determine the final capacity and resilience design.

# Interview walkthrough

## Opening (45 seconds)

The customer wants a web application that can grow with demand while keeping costs understandable. I prepared a stateless Node.js application, a Docker image definition and Kubernetes configurations on DigitalOcean. The design uses a managed load balancer and CPU-based pod scaling. I distinguish the tests I have actually run from the cloud checks still pending.

After deployment, replace the last sentence with your measured results. Do not describe pending steps as completed.

## Demonstration sequence (8-10 minutes)

1. State customer goals: growth, response time, availability and cost control.
2. Show the architecture and explain image, container, pod, node and Service.
3. Open the application and show the serving instance.
4. Show two ready replicas and their node placement.
5. Explain requests, limits, readiness and liveness probes.
6. Show the HPA target and start the bounded authorized load test.
7. Show the captured replica timeline and measured latency/error results.
8. Explain the recovery test and its limitations.
9. Compare cost scenarios and explain why pods do not directly determine the bill.
10. Finish with customer recommendations and the next decision needed.

## Questions to rehearse

Q: Why did you choose Node.js without dependencies?

A: The assignment allows a simple application. This keeps the packaging easy to inspect and focuses the demonstration on deployment, reliability and customer outcomes. Node and the base image still require patching and vulnerability review.

Q: What does the 60% HPA target mean?

A: It is average CPU utilization relative to the container CPU request. With a 100m request, 60% is approximately 60m per pod, not 60% of a worker server.

Q: Why might six pods still be slow?

A: CPU throttling, insufficient resource requests, network distance, the load generator, or downstream services may dominate. Inspect p95 latency, errors and pod CPU together before changing limits.

Q: Will adding replicas save money?

A: Adding replicas increases application capacity. Lowering replicas frees worker capacity. Worker charges decline only when billed infrastructure is removed or sized down.

Q: What happens when a node fails?

A: Surviving replicas can continue serving. Kubernetes can replace missing pods after failure detection if capacity is available. We need a distinct node failure test before claiming a recovery time.

Q: Why soft topology spreading?

A: It favors separate worker placement but allows fallback when only one worker is available. I verify placement and explain the availability tradeoff. Strict spreading could leave replacement pods pending in some failure conditions.

Q: Does a disruption budget guarantee availability?

A: It constrains supported voluntary evictions. It does not protect against every failure, direct pod deletion or a region outage.

Q: What would you add for production?

A: HTTPS, operational alerts, least-privilege access, image scanning, controlled delivery and workload-specific recovery planning. I would agree business targets before proposing more infrastructure.

# References and handoff

Assessment source: TAM_Case_Study_Assessment.docx.pdf supplied by the user. Requirements include a working containerized DOKS application, load balancing, autoscaling, cost analysis, GitHub repository and PDF or PowerPoint documentation.

## Official technical references

**Cluster creation**
https://docs.digitalocean.com/products/kubernetes/how-to/create-clusters/

**Cluster connection**
https://docs.digitalocean.com/products/kubernetes/how-to/connect-to-cluster/

**Load balancer configuration**
https://docs.digitalocean.com/products/kubernetes/how-to/configure-load-balancers/

**HPA and Metrics Server**
https://docs.digitalocean.com/products/kubernetes/how-to/set-up-autoscaling/

**Worker autoscaling**
https://docs.digitalocean.com/products/kubernetes/how-to/autoscale/

**Load balancer pricing**
https://www.digitalocean.com/pricing/load-balancers

Source check: 23 October 2026. Select current supported versions and verify the actual account quote at deployment time. The $24 worker rate in the cost model is an explicit example assumption, not a verified plan quote.

## Package contents

The ZIP includes application code, Dockerfile, five Kubernetes manifests, deployment and evidence scripts, automated tests, local results, architecture.svg, setup guide, QBR draft, interview notes and this PDF. All cloud metrics in the final customer submission must come from the deployed environment.

## Access needed to finish

A dedicated GitHub repository and an authenticated DigitalOcean environment are required for publication and live execution. The current browser could not access the DigitalOcean console. No cluster, load balancer or other paid resource was created during package preparation.